Data Mining: Classification

Fall 2026 (7761)

Dept. of Computer Science and Engineering

Instructor: Ting-Wei Lin

2026/11/06, Week 9

國立中興大學
NATIONAL CHUNG HSING UNIVERSITY

Last Week

- What is classification?
  - Given a collection of recordings: captures the relationship between attributes and class labels.
  - Goal: given a new object with an unknown class label, predict which class it belongs to.
- Classification Methods
  - Decision Trees: information gain, gain ratio, and gini impurity.
  - Naïve Bayes Classifier: each attribute is conditionally independent.

This week

- Lazy Learning
  - K-Nearest-Neighbors
  - Case-Based Reasoning
- Linear Classifiers
  - Logistic Regression
  - Support Vector Machines
- Ensemble Classifiers
  - Bagging, Boosting, AdaBoost, Random Forest

Lazy Learning

Classifying a new instance using nearby training instances

Eager Learning vs. Lazy Learning

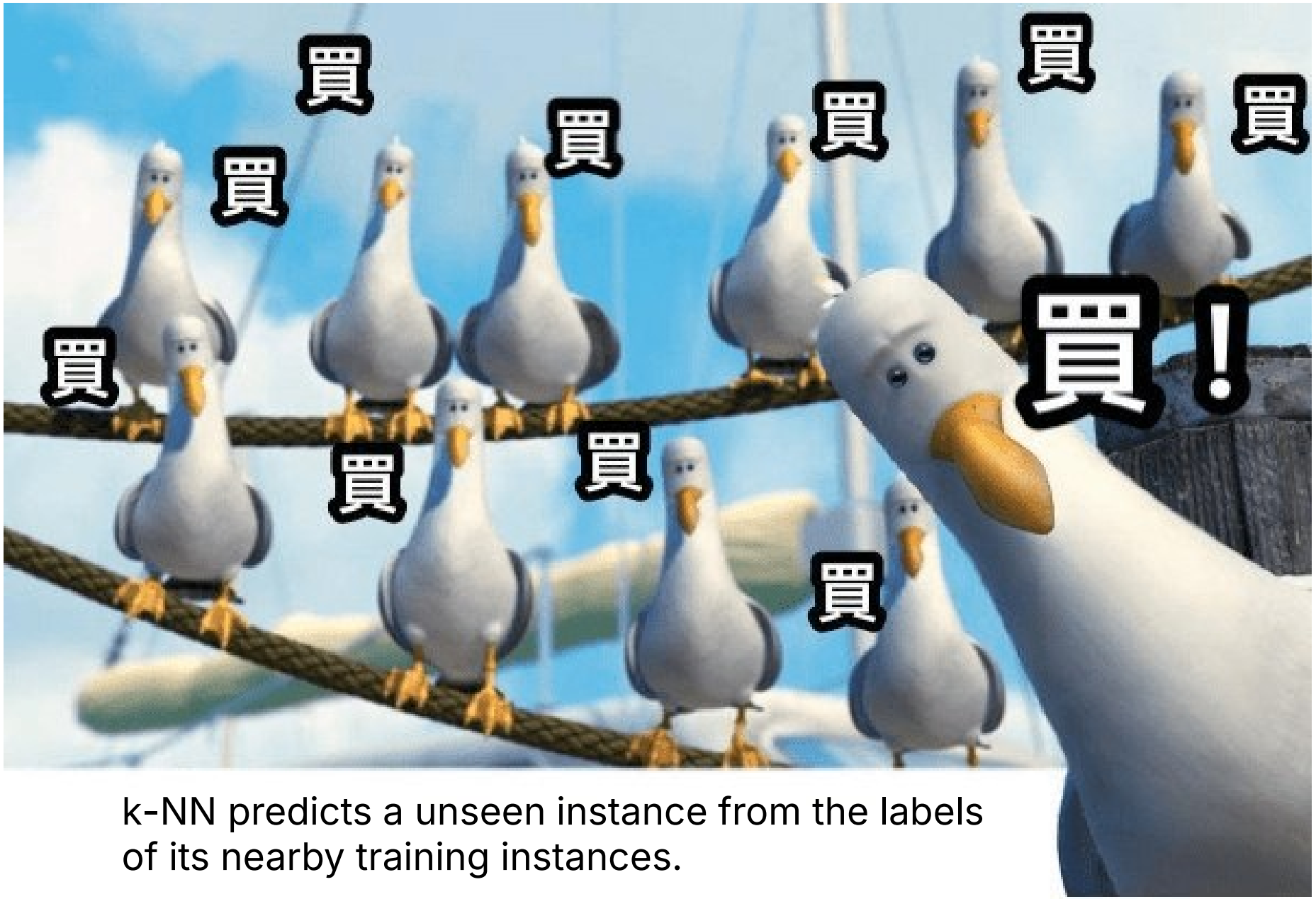
- Eager learning (decision tree, naïve bayes)
  - Construct a classification model from the training data before receiving unseen tuples to classify.
  - The learned model as being ready and eager to classify previously unseen tuples
- Lazy learning (instance-based learning)
  - Store the training data (with little preprocessing) and delays generalization until a query tuple arrives.
  - Defers most generalization until a unseen arrives, and predicts its label from similar stored instances.

Lazy Learning

- Trade-offs
  - Less computation during training, but more computation at prediction time.
  - May require substantial storage for training instances.
- Typical approaches
  1. k-Nearest Neighbor (k-NN)
     - Instances are represented as feature vectors in a metric space.
  2. Case-Based Reasoning (CBR)
     - Retrieves similar past cases and adapts their solutions to the new case.

Lazy Learning:
① k-Nearest Neighbor

Predict from neighbor.



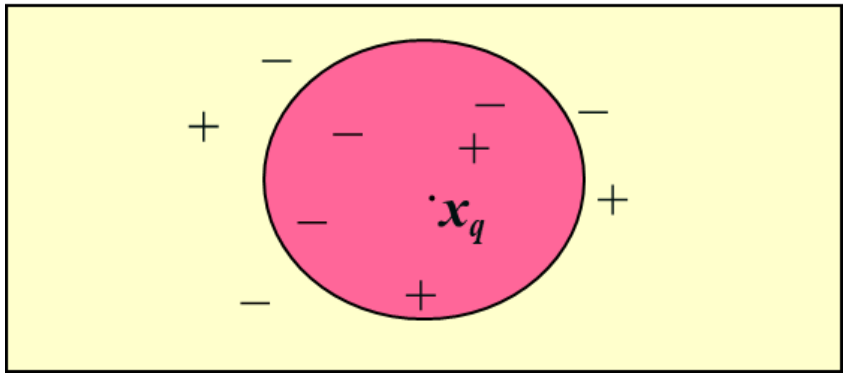
k-NN predicts a unseen instance from the labels of its nearby training instances.

k-Nearest Neighbors (k-NN)

- Each instance is represented as a point in a d-dimensional feature space.
  - The nearest neighbors are determined using a distance metric, such as Euclidean distance.

$$dist(X_1, X_2) = \sqrt{\sum_{i=1}^{d}(x_{1i} - x_{2i})^2}$$

  - For a nominal attribute, a simple matching distance assigns 0 to identical values and 1 to different values.



Feature Scaling Matters in k-NN

- k-NN relies on distances between feature vectors.
- Features with larger numeric ranges can dominate the distance.
- Standardize numeric features using training-set statistics:

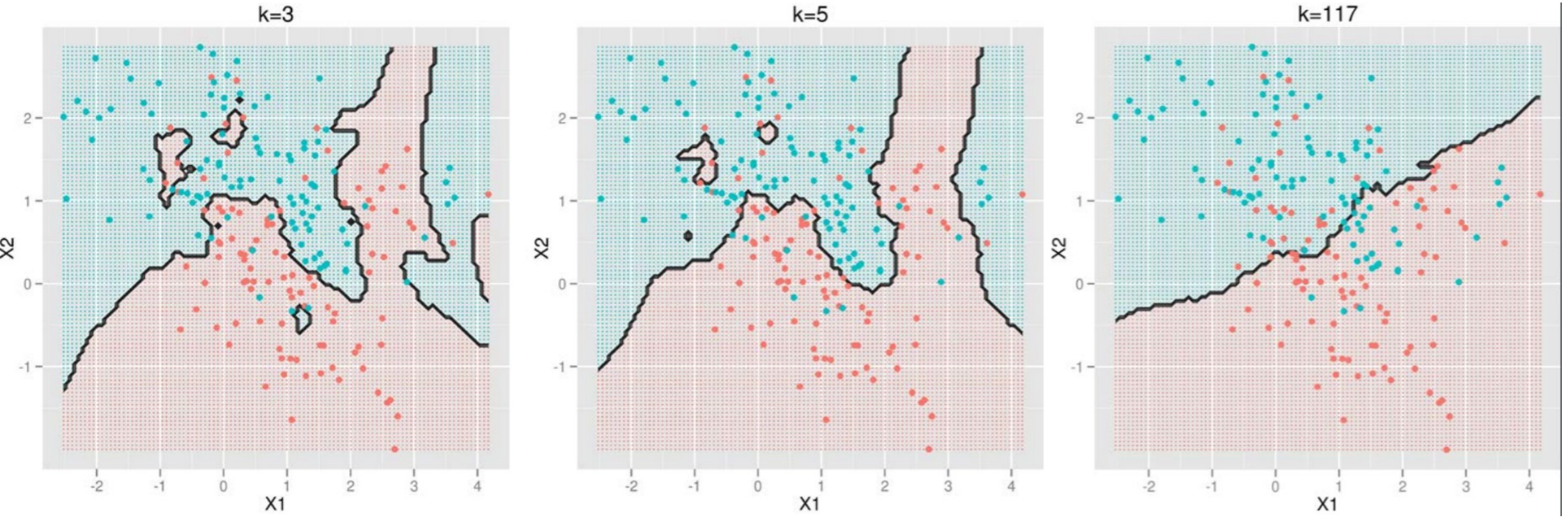
$$z_k = \frac{x_k - \mu_k}{\sigma_k}$$

  - Fit μ_k and σ_k on the training set only, then use them to transform validation, test, and unseen data.

Prediction with k-Nearest Neighbors

- For a unseen data
  - Computer distance → Selection the k nearest training data → Vote or Average
  - Classification: choose the majority class among the k nearest training instances.
  - Regression: Predict the mean target value of the k nearest neighbors

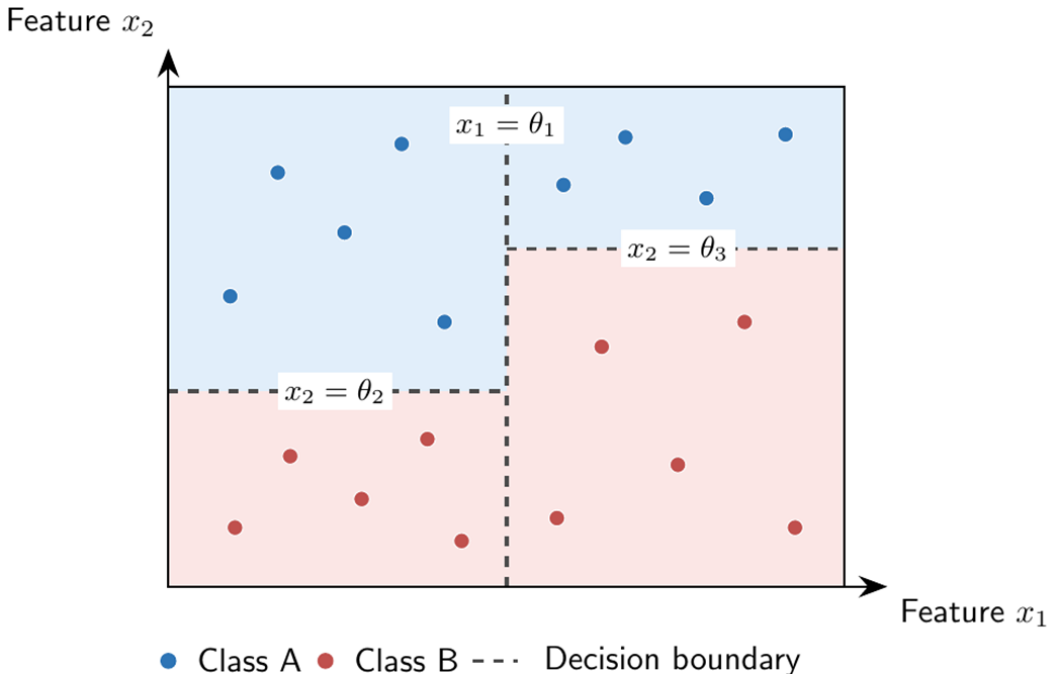
How Does k Affect the Decision Boundary?



Source: http://scott.fortmann-roe.com/docs/BiasVariance.html

Decision Boundaries in Classification

- A classifier divides the feature space into regions assigned different predicted class labels.
- The boundaries between these regions are called decision boundaries.



Example: A decision tree creates axis-aligned decision boundaries through threshold-based splits.

Bias–Variance Trade-off in Classification

- Small k: overfitting (high variance, low bias)
- Large k: over smoothing (high bias, low variance)
  - Bias: Systematic error caused by overly restrictive model assumptions.
    - The decision boundary is too simple.
    - The model tends to underfit.
  - Variance: Sensitivity of a model to the particular training set.
    - The decision boundary can change substantially when the training data changes.
    - The model tends to overfit.

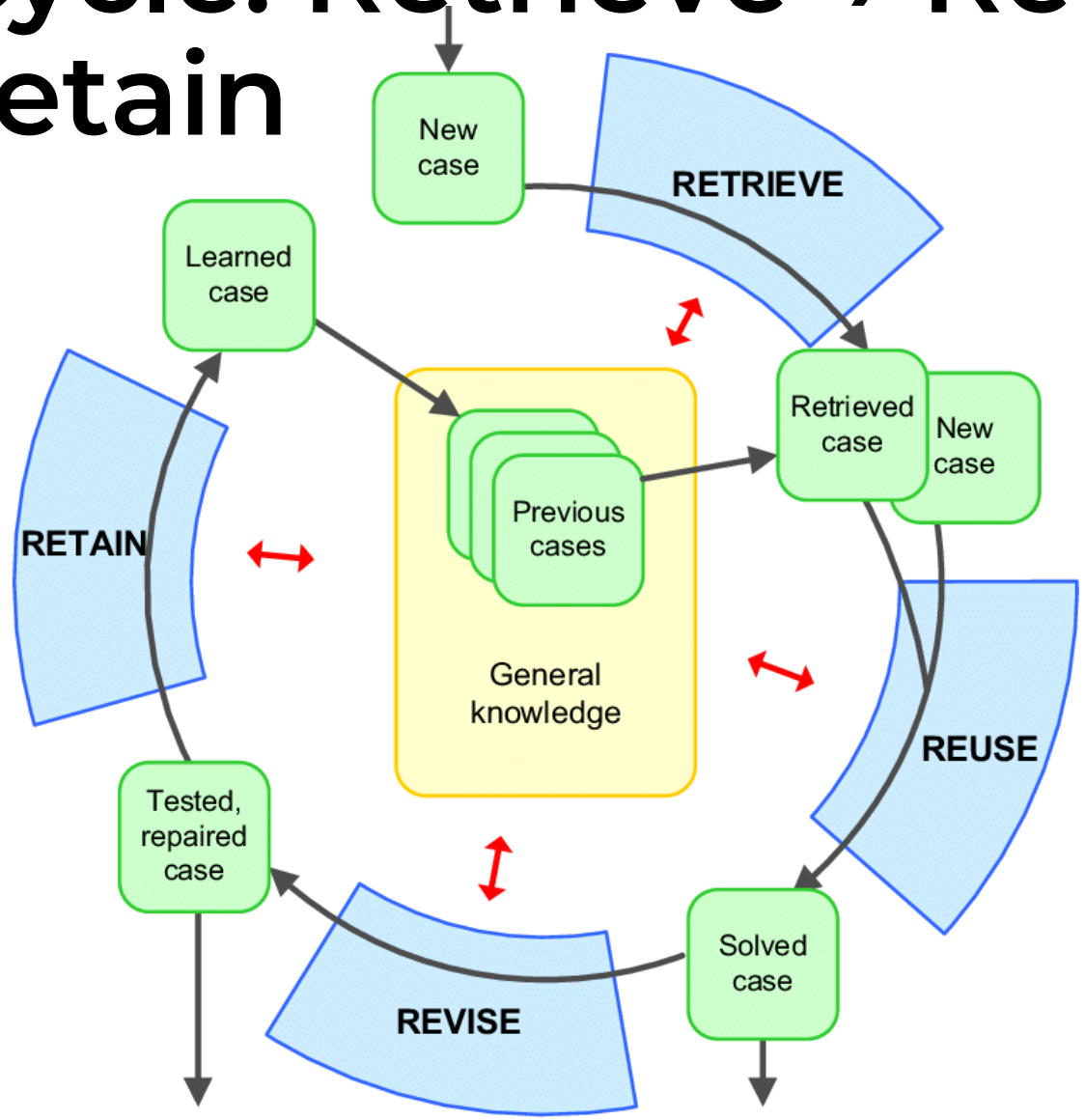
Lazy Learning:
② Case-Based Reasoning

Retrieving and adapting past cases to solve new problems

Case-Based Reasoning (CBR): The 4R Cycle

- A case base stores past cases, including their problem descriptions, solutions, and outcomes.
- To solve a new problem, CBR follows four steps (4R):
  a. Retrieve: Find past cases that are most similar to the new case.
  b. Reuse: Adapt a retrieved solution to the new case.
  c. Revise: Evaluate and, if necessary, repair the proposed solution.
  d. Retain: Store the new experience for future use.
- Key challenge: Define a meaningful similarity measure and an appropriate adaptation strategy.

The CBR Cycle: Retrieve → Reuse → Revise → Retain



Source: https://ithelp.ithome.com.tw/m/articles/10294286

Example: Making Blueberry Pancakes

a. Retrieve
   - Fred wants to prepare blueberry pancakes.
   - He retrieves a past case about making plain pancakes.
b. Reuse
   - He adapts the recipe by adding blueberries.
c. Revise
   - Adding blueberries to the butter turns it blue, so he revises the procedure.
d. Retain
   - He stores the improved blueberry-pancake recipe as a new case.

Unlike k-NN, CBR may adapt a retrieved solution instead of simply voting among neighbors.

Linear Classifiers

Logistic Regression, Support Vector Machines

What are Linear Classifiers?

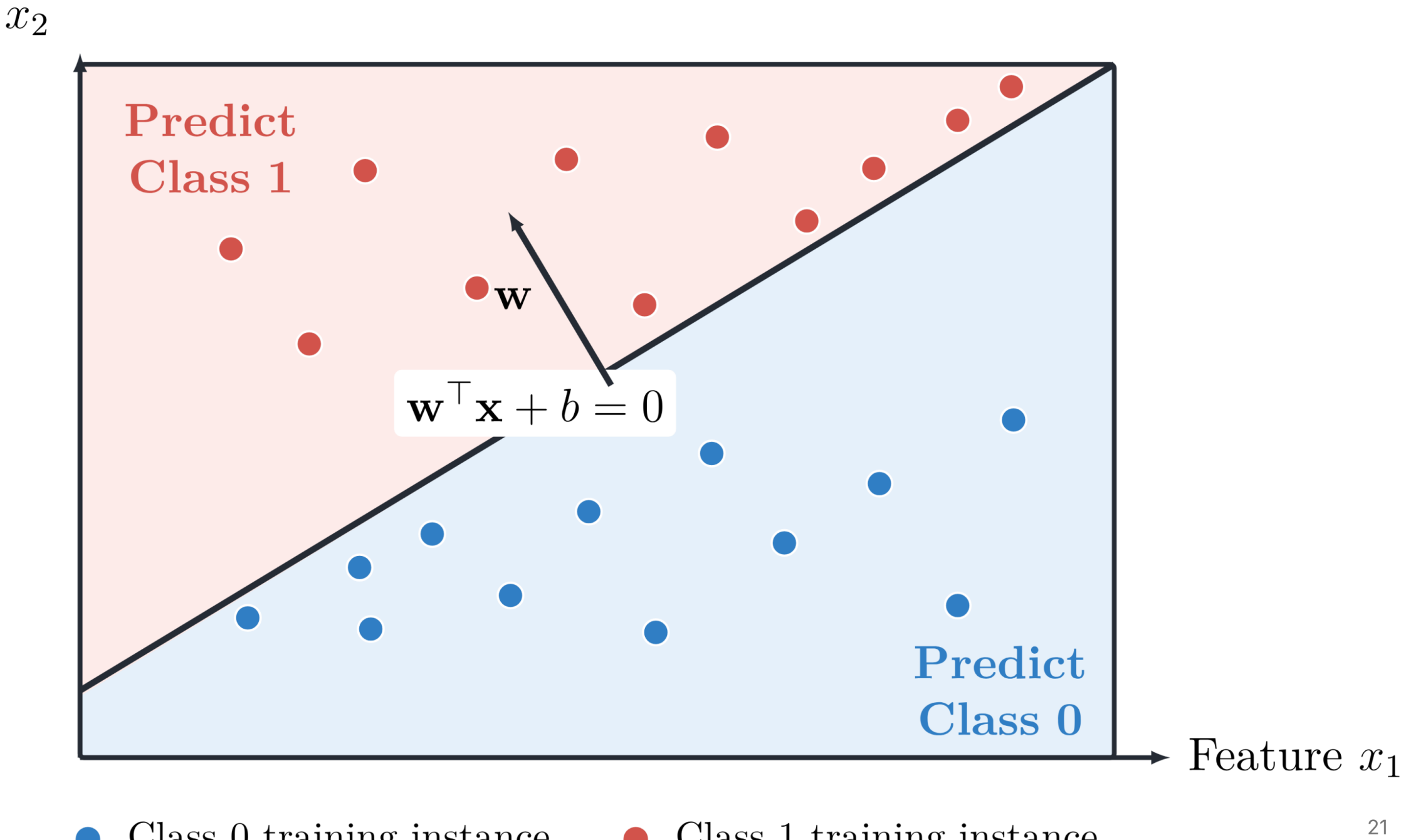
- A linear classifier is a supervised machine learning algorithm that predicts a data point's class by combining its input features with a linear function.

$$f(x) = w^T x + b$$

- Predicts a class according to which side of the decision boundary the instance lies on:

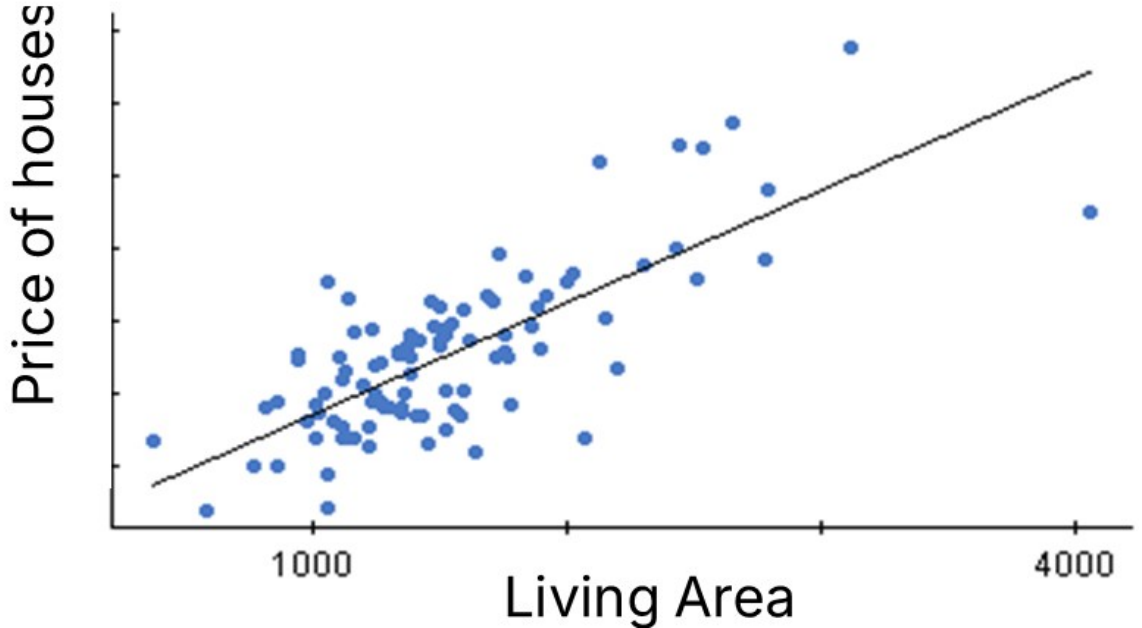
$$\hat{y} \begin{cases} 1, & f(x) \geq 0 \\ -1, & f(x) < 0 \end{cases}$$

  - The decision boundary is: $f(x) = w^T x + b = 0$
  - In 2D, it is a line. In higher dimensions, it is a hyperplane.


x_2
Predict
Class 1
$\mathbf{w}$
$\mathbf{w}^\top \mathbf{x} + b = 0$
Predict
Class 0
Feature x_1
Class 0 training instance
Class 1 training instance

Linear Regression Refresh

- Linear regression predicts a continuous target value from one or more input attributes.
  - Predict the housing price based on the living area



Linear Regression (cont.)

- Learn a linear function that maps the n input attributes x_i to the output variable

$$y_i: \widehat{y_i} = w^T x_i + b = \sum_{j=1}^{p} w_j x_{i,j} + b$$

  - Each example is represented by p attributes: $x_i = (x_{i,1}, \dots x_{i,p})$
  - Continuous output value: y_i
- Assumes that the output value is a linear weighted summation of the p input attribute values, offset by the bias scalar b
  - w_j tells how important the corresponding attribute $x_{i,j}$ in predicting y_i
- The weight vector w and bias b are the model parameters learn by data

Linear regression (cont.)

- Object: learn the best weight vector w and the best bias scalar b from the training data to make best prediction
  - $\widehat{y_i} = w^T x_i + b$: $\widehat{y_i}$ is as close as possible to y_i

$$L(w, b) = \sum_{i=1}^{n} (y_i - \widehat{y_i})^2 = \sum_{i=1}^{n} \left(y_i - (w^T x_i + b)\right)^2$$

$$(\widehat{w}, \widehat{b}) = \underset{w,b}{arg\min}\, L(w, b)$$

- Ways to solve this optimization
  - Closed-form solution: $\theta = (X^T X)^{-1} X^T y$
  - Gradient descent: $\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$

Optional: Closed-form solution

- One input feature

$$\theta = (X^T X)^{-1} X^T y \rightarrow w = \frac{\sum_{i=1}^{n} x_i (y_i - \bar{y})}{\sum_{i=1}^{n} x_i^2 - \frac{1}{n}\left(\sum_{i=1}^{n} x_i\right)^2}, b = \frac{1}{n}\sum_{i=1}^{n}(y_i - w x_i)$$

Attribute	1	3	5	7
Output	4	10	14	16

$$\bar{y} = \frac{4 + 10 + 14 + 16}{4} = 11$$

$$w = \frac{1(4 - 11) + 3(10 - 11) + 5(14 - 11) + 7(16 - 11)}{(1^2 + 3^2 + 5^2 + 7^2) - \frac{1}{4}(1 + 3 + 5 + 7)^2} = 2$$

$$b = \frac{(4 - 2 \times 1) + (10 - 2 \times 3) + (14 - 2 \times 5) + (16 - 2 \times 7)}{4} = 3$$

Linear Classifiers:
① Logistic Regression

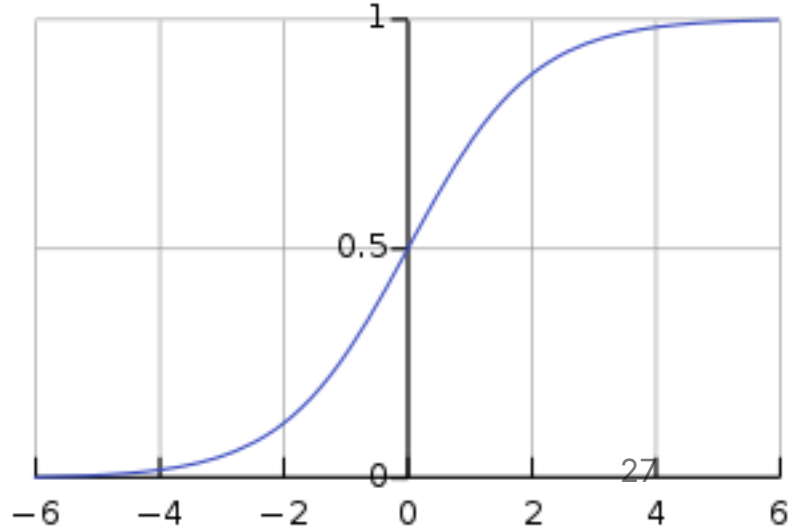
Logistic regression

- How can a linear model solve a classification problem?
  - For binary classification, we model the probability that an instance belongs to Class 1

$$P(Y = 1|x)$$

  - First compute a linear score: $z = w^T x + b$
  - Apply the sigmoid (logistic) function to transform this score into a probability:

$$P(Y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



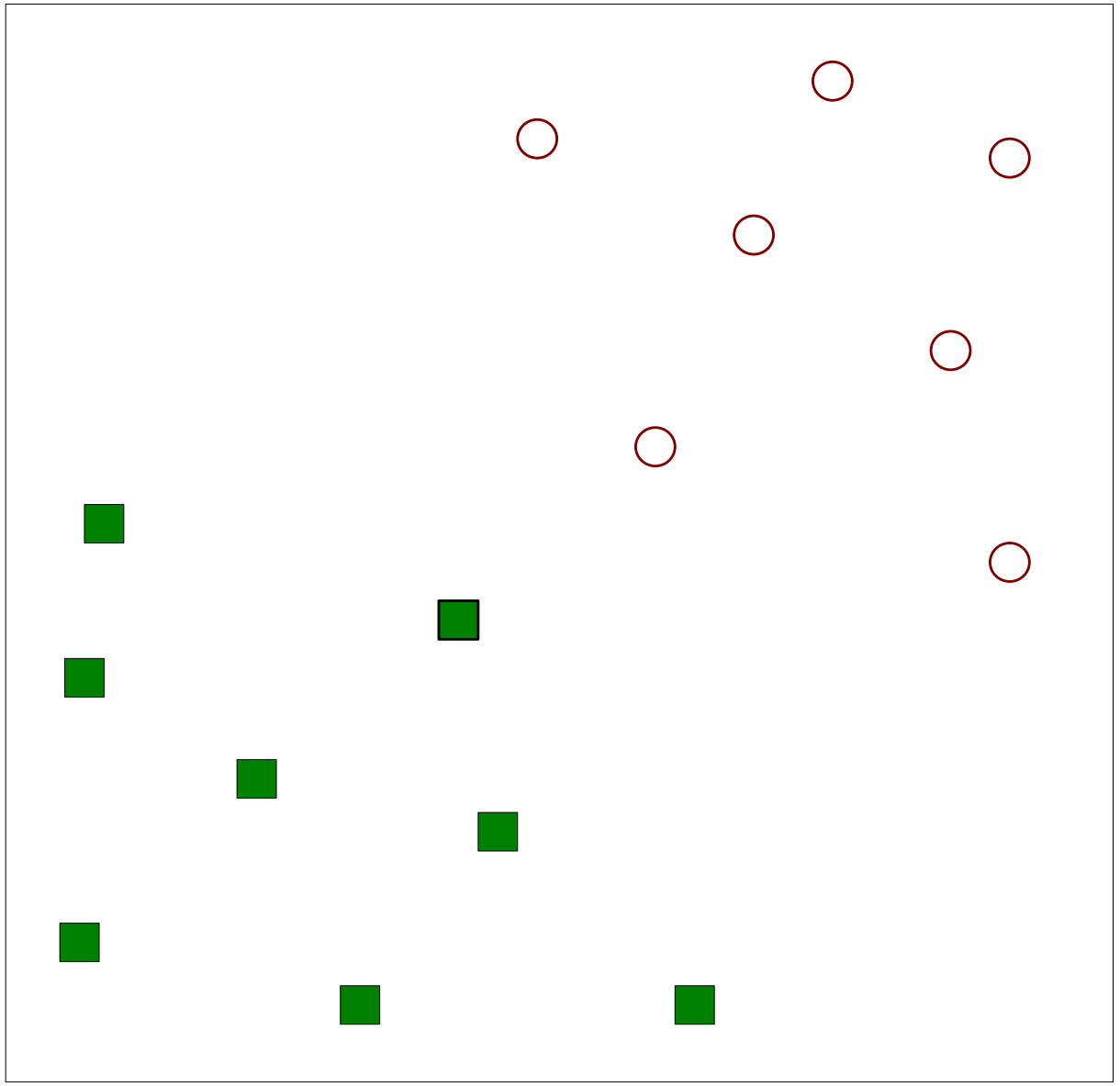
Logistic regression (cont.)

- Probability that: $\widehat{p_i} = P(Y_i = 1 \,|x_i) \;=\; \sigma(w^T x_i + b)$
  - $If\ \widehat{p_i} \geq 0.5$: positive ($\widehat{y_i}$=1)
  - $If\ \widehat{p_i} < 0.5$: negative ($\widehat{y_i}$=−1)
- Maximum Likelihood Estimation
  - Find the parameters that make the observed training data most likely:
    - $P(Data \mid Model)$
  - $L(w, b) = \Pi_i p_i^{y_i}(1 - p_i)^{1-y_i}$ = $\Pi_i \left(\frac{\exp(w^T x_i + b)}{1+\exp(w^T x_i + b)}\right)^{y_i} \left(\frac{1}{1+\exp(w^T x_i + b)}\right)^{1-y_i}$
  - Maximize the log-likelihood using gradient ascent

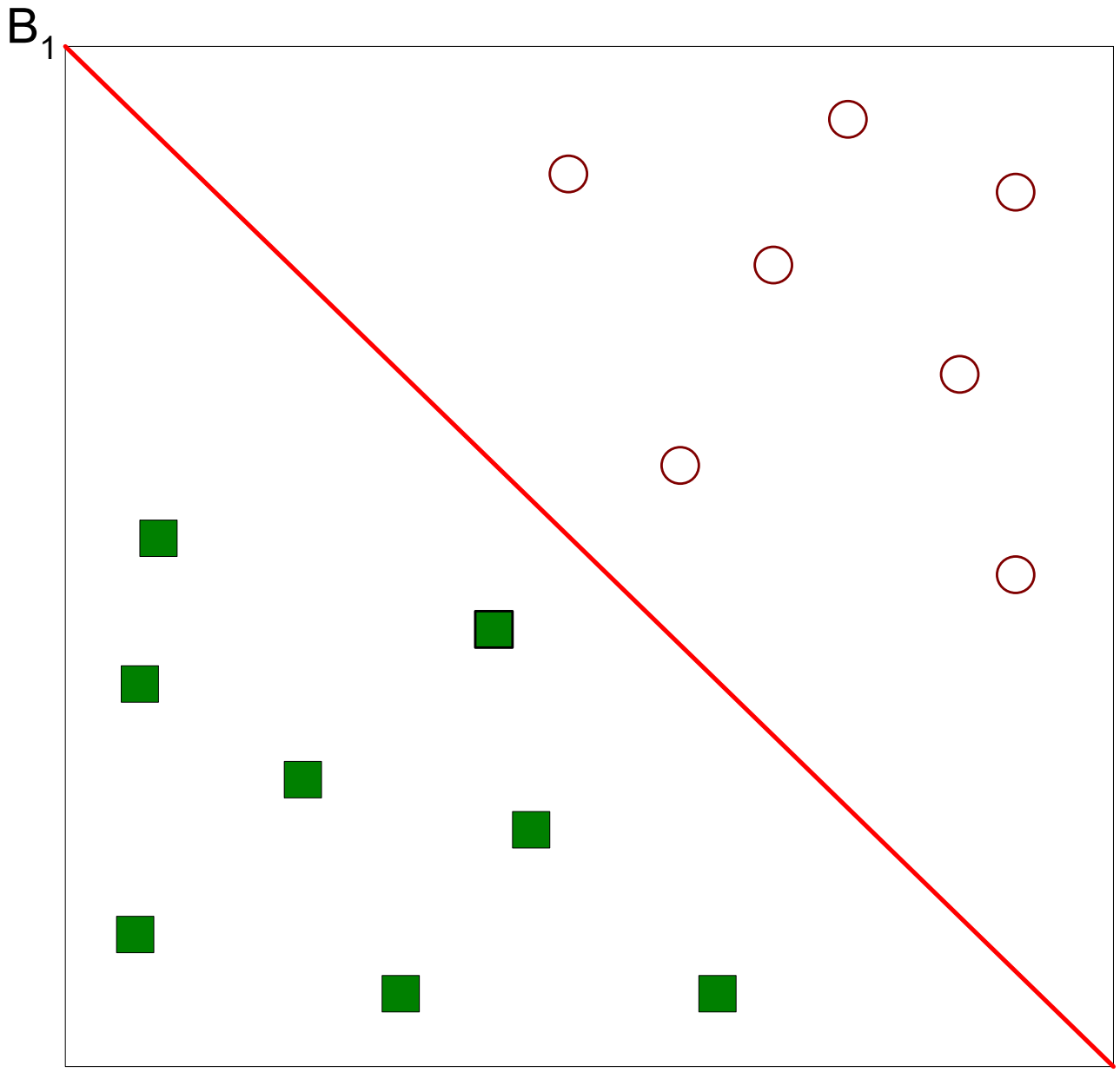
Linear Classifiers:
② Support Vector Machines

Support Vector Machines

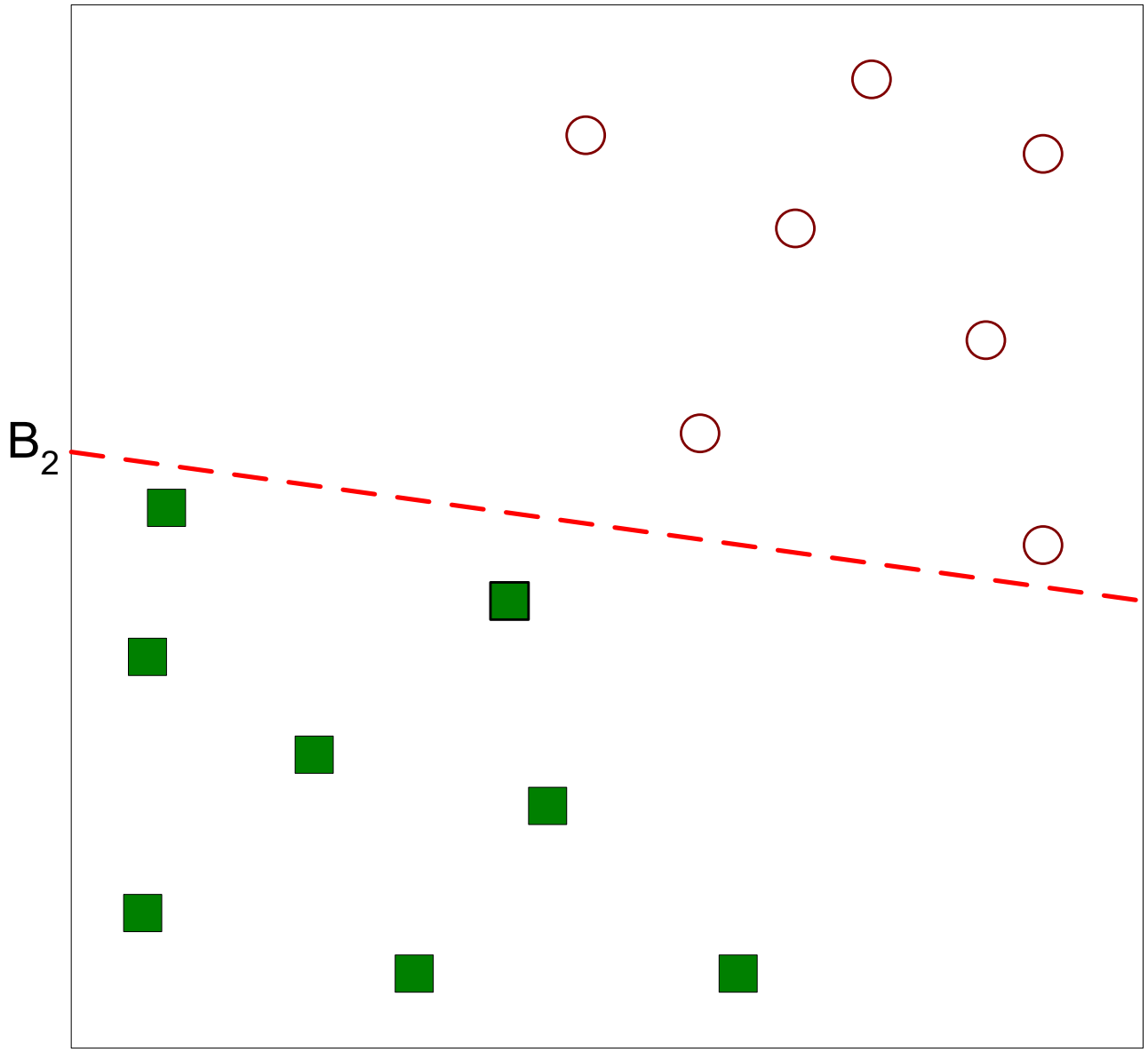
- A support vector machine (SVM) is a discriminative model that learns a decision boundary to separate classes in the feature space.
  - The boundary can be linear or nonlinear.
- With an appropriate nonlinear mapping induced by a kernel, data that are not linearly separable in the original feature space may become linearly separable in a higher-dimensional feature space.



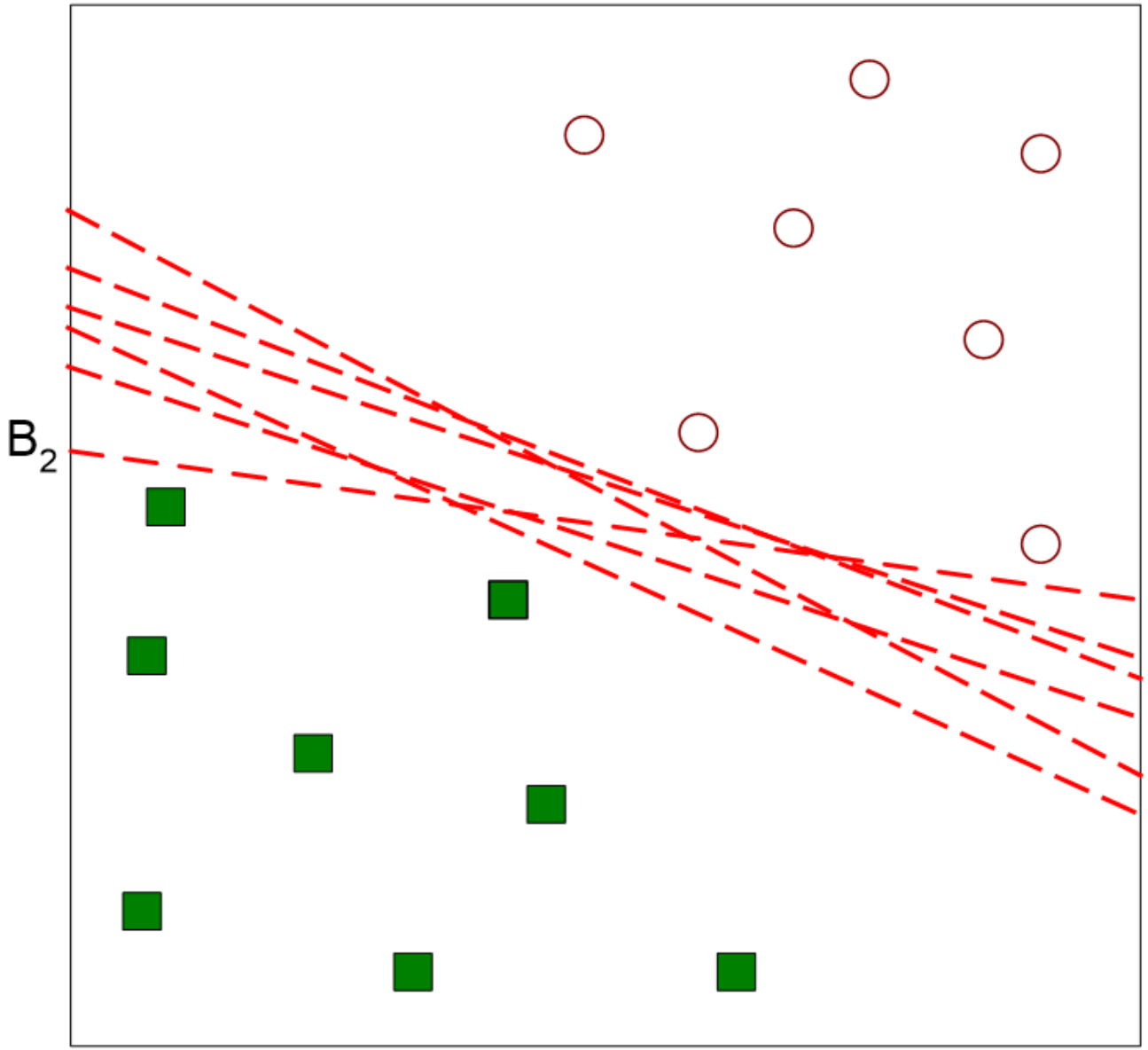
Find a linear decision boundary that separates the two classes.



One Possible Solution

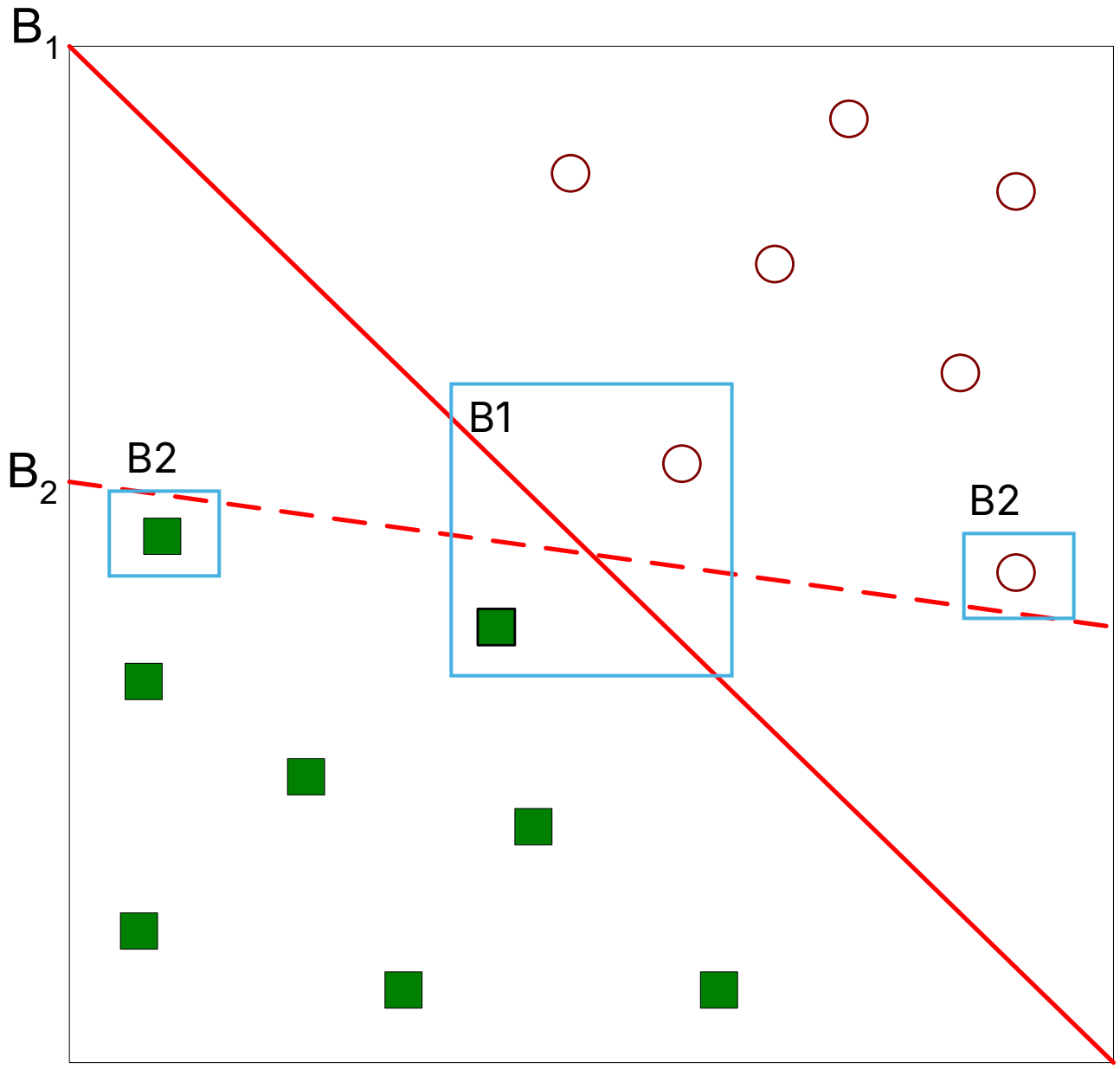


Another possible solution



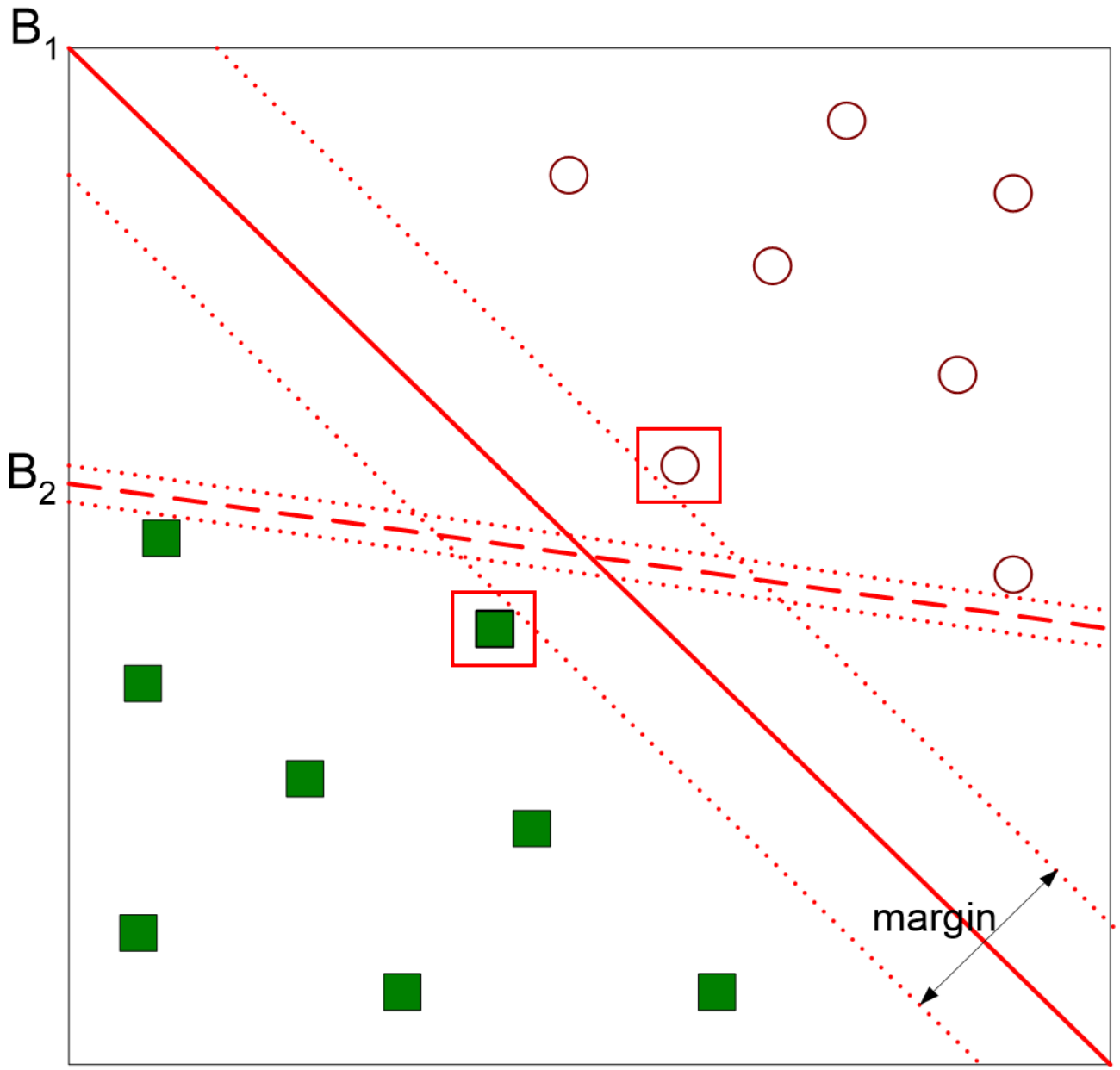
There are many hyperplanes that can separate the training samples.

Let’s look at the points close to the hyperplane

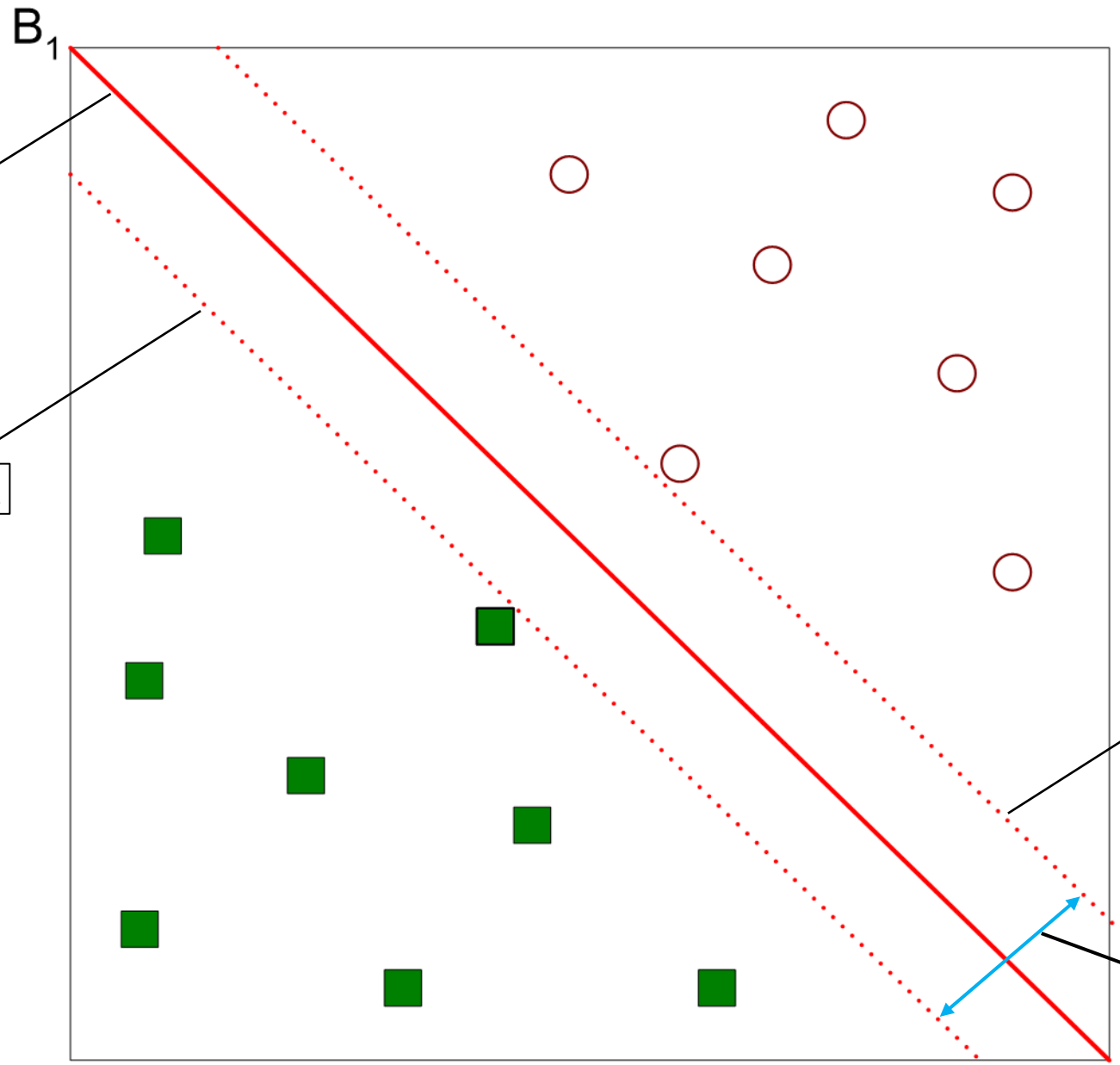


Which one is a better hyperplane?

Support vectors: the data points that lie closest to the decision boundary



We want to find hyperplane maximizes the margin ⇒ B1 is better than B2



$w^T x + b = 0$

$w_1 x_1 + w_2 x_2 + b = 0$

$w^T x + b = -1$

$w^T x + b = +1$

$Margin = \frac{2}{||w||}$

$||w||$: Euclidean norm of w

Hard-margin

$$(x_i, y_i) \Rightarrow y_i = \begin{cases} 1, if\ w^T x_i + b \geq 1 \\ -1, if\ w^T x_i + b \leq -1 \end{cases} \Rightarrow y_i(w^T x_i + b) \geq 1, y \in \{-1, +1\}$$

Linear SVM

- Objective is to maximize the margin: $\frac{2}{||w||}$
- Which is minimize $\underset{w,b}{\operatorname{argmin}} \frac{\|w\|^2}{2}$
  - Subject to the following constraints, for each training points (x_i, y_i)

$$y_i(w^T x_i + b) \geq 1, i = 1, \ldots, N$$

  - This is a constrained optimization problem. We solve it using:
    - Lagrange multipliers
    - Karush-Kuhn-Tucker (KKT) conditions

Optional: Lagrange Multiplier

- Lagrangian: $L(w, b, \lambda) = \frac{1}{2}\|w\|^2 - \sum_i \lambda_i [y_i(w^T x_i + b) - 1]$

	x_1	x_2	y	Lagrange Multiplier
support vector	0.3858	0.4687	1	65.5261
support vector	0.4871	0.611	–1	65.5261
	0.9218	0.4103	–1	0
	0.7382	0.8936	–1	0
	0.1763	0.0579	1	0
	0.4057	0.3529	1	0
	0.9355	0.8132	–1	0
	0.2146	0.0099	1	0

$$\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_i \lambda_i y_i x_i$$

- $w_1 = 65.5261 \times 1 \times 0.3858 + 65.5261 \times -1 \times 0.4871 = -6.6377$
- $w_2 = 65.5261 \times 1 \times 0.4687 + 65.5261 \times -1 \times 0.6110 = -9.3243$
- $b^1 = 1 - w^T x = 1 - (-6.6377) \times 0.3858 - (-9.3243) \times 0.4687 = 7.9311$
- $b^2 = -1 - w^T x = -1 - (-6.6377) \times 0.4871 - (-9.3243) \times 0.6110 = 7.9303$
- $b = \frac{b^1 + b^2}{2} = 7.9307$
- $f(x) = -6.6377x_1 - 9.3243x_2 + 7.9307$

Prediction with a SVM

- For a new sample x, compute the decision score:

$$f(x) = w^T x + b$$

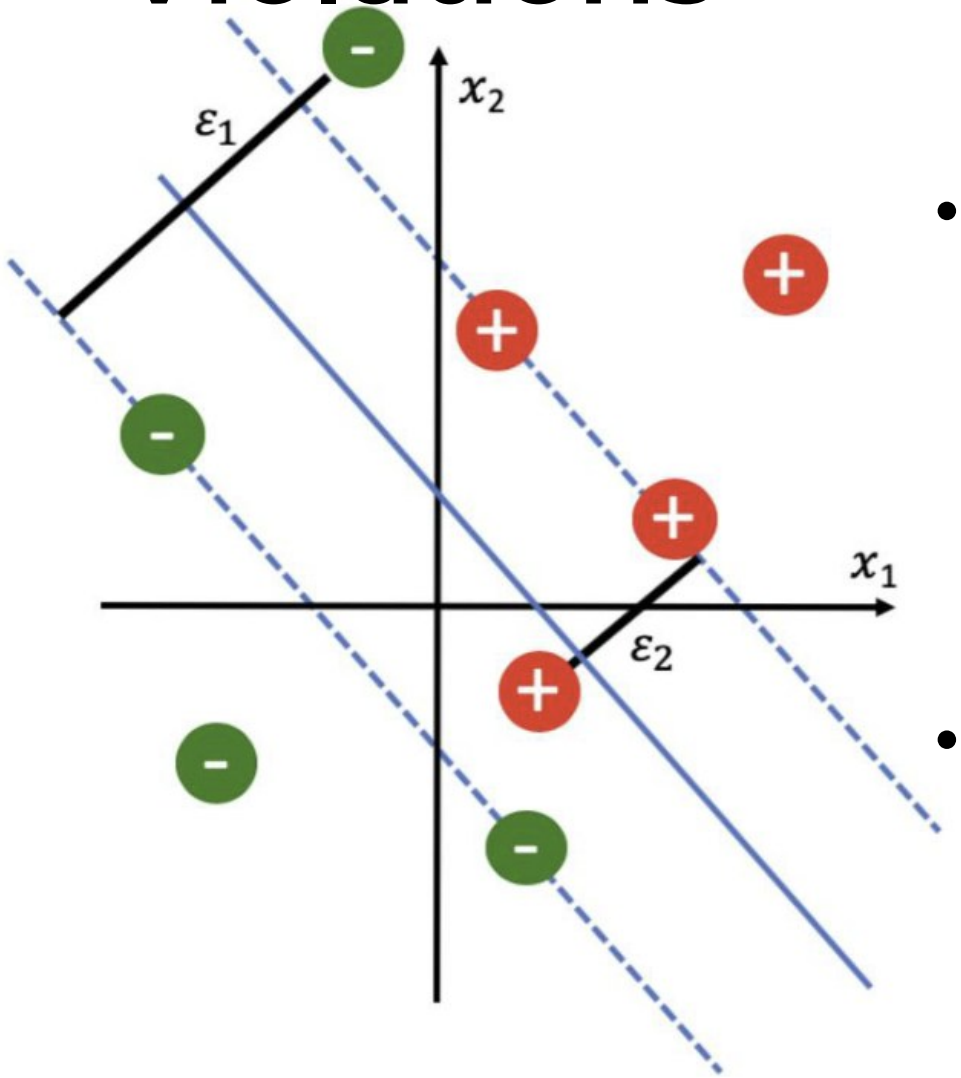
- The predicted class is determined by the sign of the score:

$$\hat{y} = \begin{cases} 1, & f(x) \geq 0 \\ -1, & f(x) < 0 \end{cases}$$

Range	Location	Prediction
$f(x) \geq +1$	Positive side, on or outside the margin	$+1$
$0 \leq f(x) < +1$	Positive side, inside the margin	$+1$
$-1 < f(x) < 0$	Negative side, inside the margin	-1
$f(x) \leq -1$	Negative side, on or outside the margin	-1

In a hard-margin SVM, training samples are not allowed to fall inside the margin.
A new unseen sample may still fall there and is classified by which side of $f(x)$ it lies on.

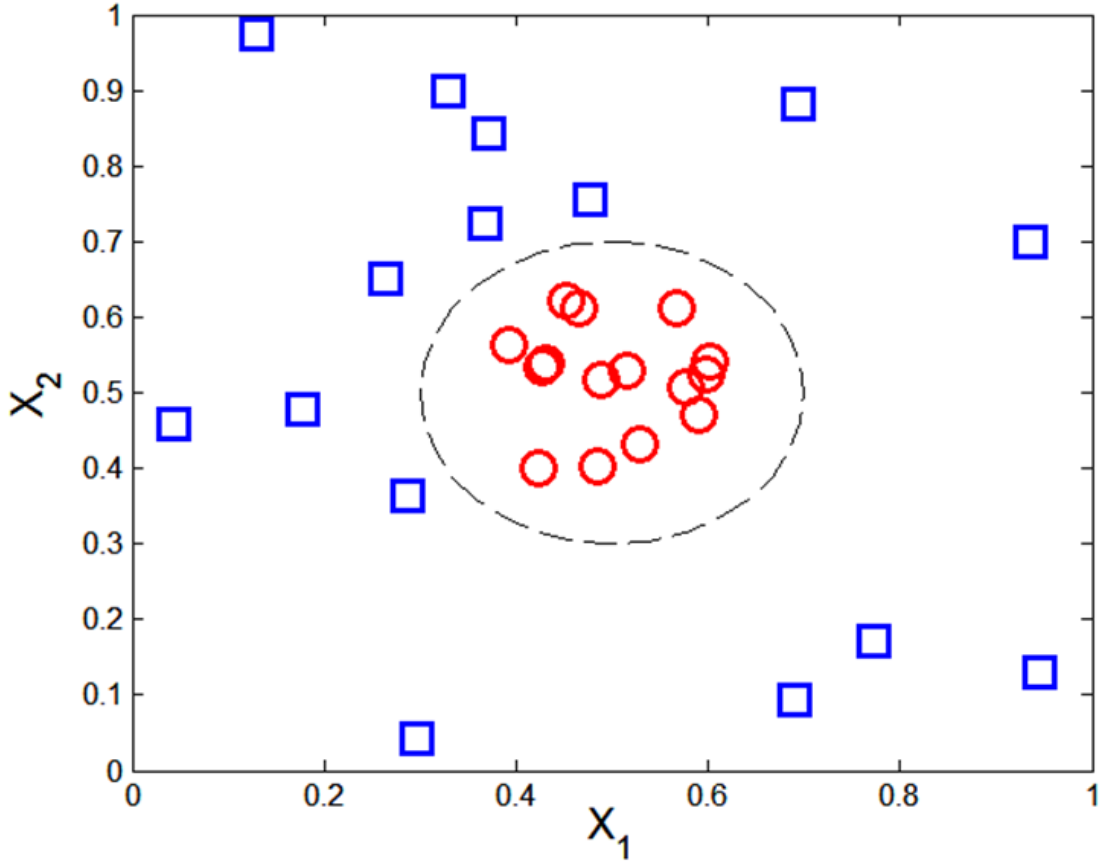
Soft-margin SVM: Allowing Margin Violations

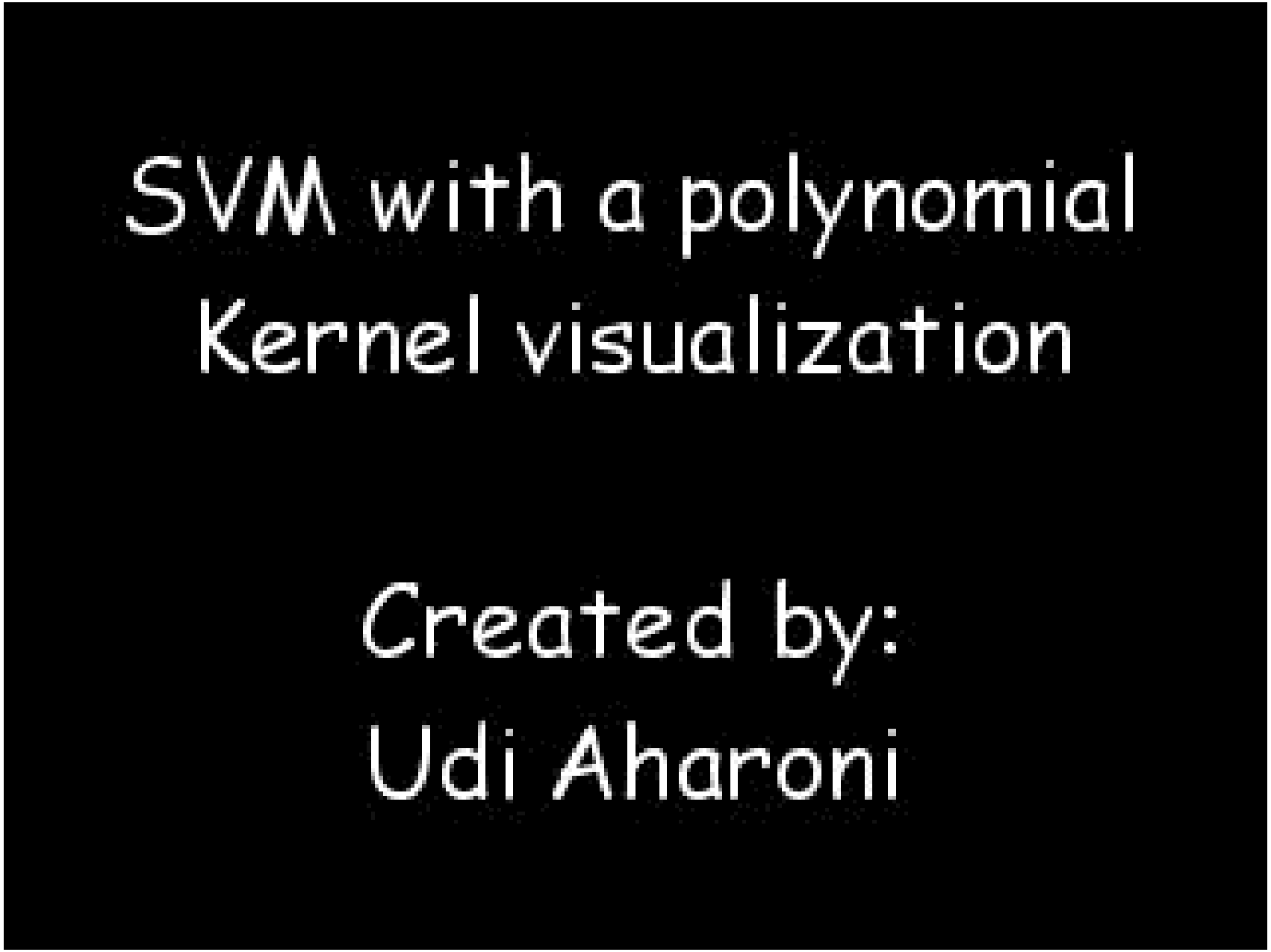


- $\underset{w,b,\xi}{\operatorname{argmin}} \frac{\|w\|^2}{2} + C \sum_i^N \xi_i$
  - C controls the trade-off between a large margin and penalty for margin violations.
  - Larger C penalizes violations more heavily
  - Smaller C allows more violations in exchange for a wider margin.
- Soft-margin SVM balances:
  - Large margin
  - Low classification error

Non-Linear SVM

- What if decision boundary is not linear?

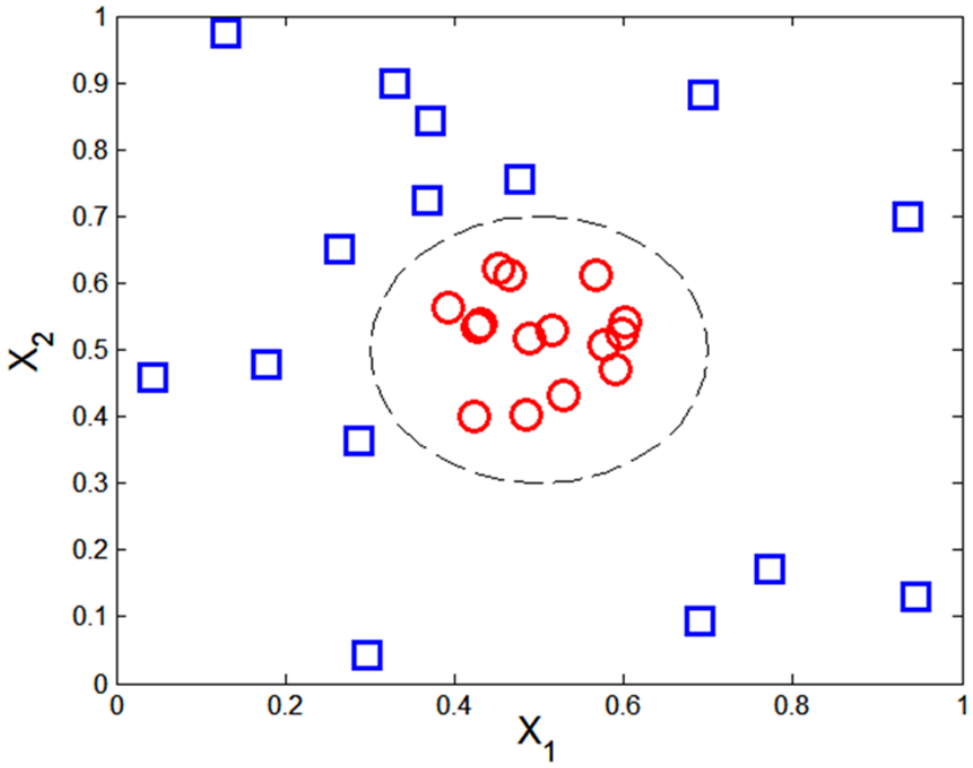




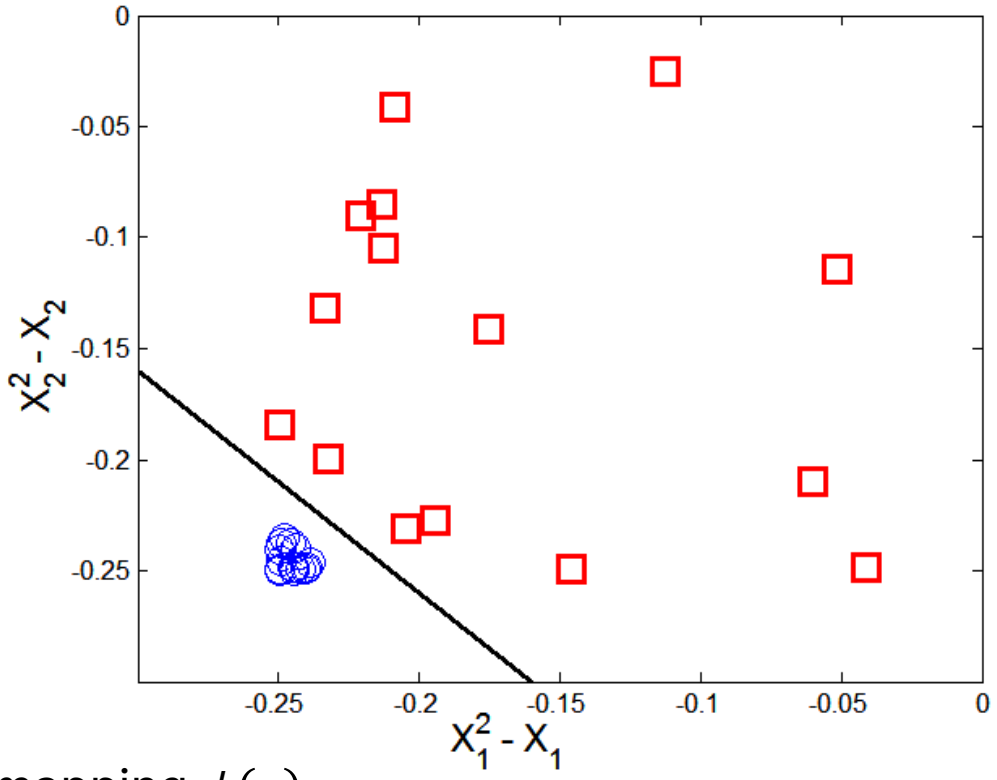
https://www.youtube.com/watch?v=3liCbRZPrZA

Non-Linear SVM (cont.)

- Transform data into higher dimensional space



$$\phi(x) = \begin{bmatrix} x_1^2 - x_1 \\ x_2^2 - x_2 \end{bmatrix}$$



Apply a nonlinear feature mapping $\phi(x)$

Non-Linear Feature Mapping

- If we have a 3D input vector $X = (x_1, x_2, x_3)$
  - Example feature mapping : $\phi(\mathrm{X}) \rightarrow (x_1, x_2, x_3, x_1^2, x_1x_2, x_1x_3)$
  - $w_1x_1 + w_2x_2 + w_3x_3 \rightarrow w_1x_1 + w_2x_2 + w_3x_3 + w_4x_1^2 + w_5x_1x_2 + w_6x_1x_3$
  - $y_i(w^T x_i + b) \geq 1 \rightarrow y_i(w^T \phi(x_i) + b) \geq 1$
- If $\phi(\mathrm{X})$ maps into very high (even infinite) dimension
  - Explicitly computing $\phi(\mathrm{X})$ can be expensive or impossible when the feature space is very high-dimensional.

Kernel function

- For a test sample z, prediction requires inner products in the feature space

$$f(z) = w^T\phi(z) + b = \sum_i \lambda_i y_i (\phi(x_i)^T\phi(z)) + b, w = \sum_i \lambda_i y_i \phi(x_i)$$

- Kernel function
  - Compute the inner product in some (possibly high-dimensional) feature space $\mathrm{K}(\mathrm{x}_i, z) = \phi(x_i)^T\phi(z)$
  - We do not need to explicitly compute the mapped representations x_i and z

$$f(z) = w^T\phi(z) + b = \sum_i \lambda_i y_i K(x_i, z) + b$$

Example: Second-degree Polynomial Kernel

$x = (x_1, x_2), z = (z_1, z_2)$ $\phi(x) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$

$\phi(x)^T\phi(z)$

- $\phi(x) = \left(x_1^2, \sqrt{2}x_1x_2, x_2^2\right)$
- $\phi(z) = (z_1^2, \sqrt{2}z_1z_2, z_2^2)$

- Dot product

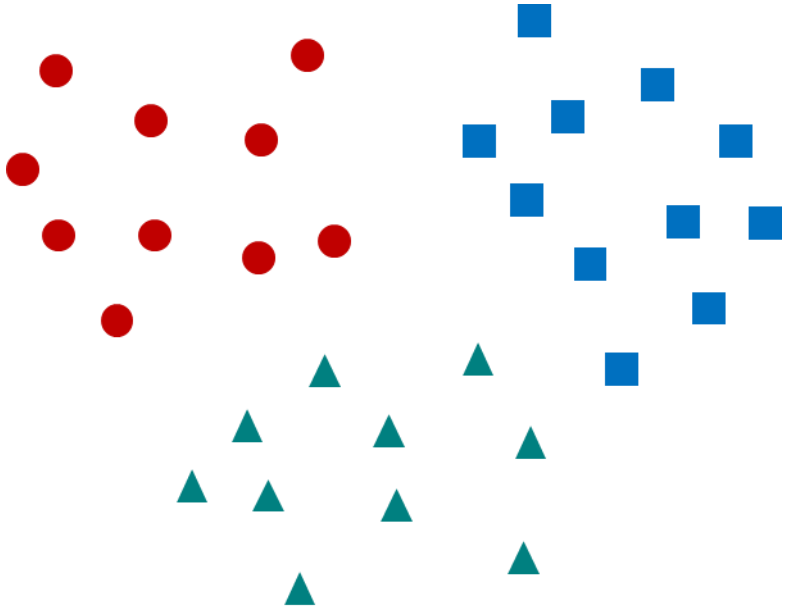
$$x_1^2z_1^2 + 2x_1x_2z_1z_2 + x_2^2z_2^2 = (x_1z_1 + x_2z_2)^2 = (x^Tz)^2$$

$$Kernel(x, z) = (x^Tz)^2 = \phi(x)^T\phi(z)$$

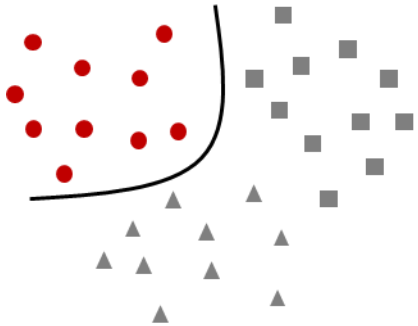
Kernel trick

- Kernel Trick
  - Compute inner products in a feature space without explicitly computing the feature mapping.
- Linear kernel: $K(x, z) = x_i^T z$
- Polynomial kernel: $K(x, z) = (x^T z + c)^d$
- RBF (Gaussian) kernel: $K(x_i, z) = e^{-\frac{\|x-z\|^2}{2\sigma^2}} (\gamma = \frac{1}{2\sigma^2})$

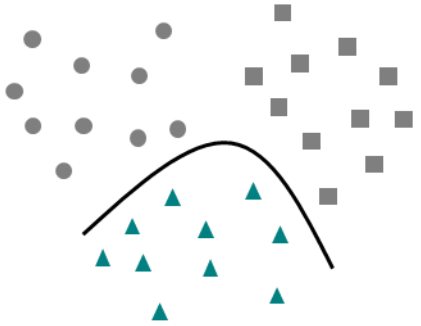
Multi-class in SVM



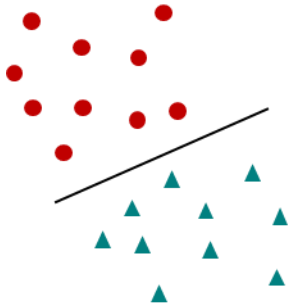
N classes

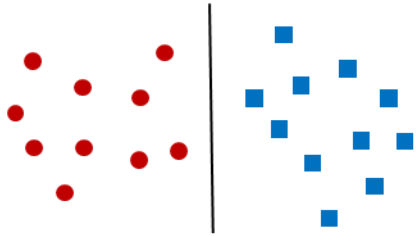


One-vs-Rest

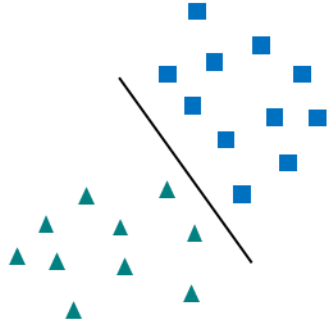


Requires N classifiers





One-vs-One



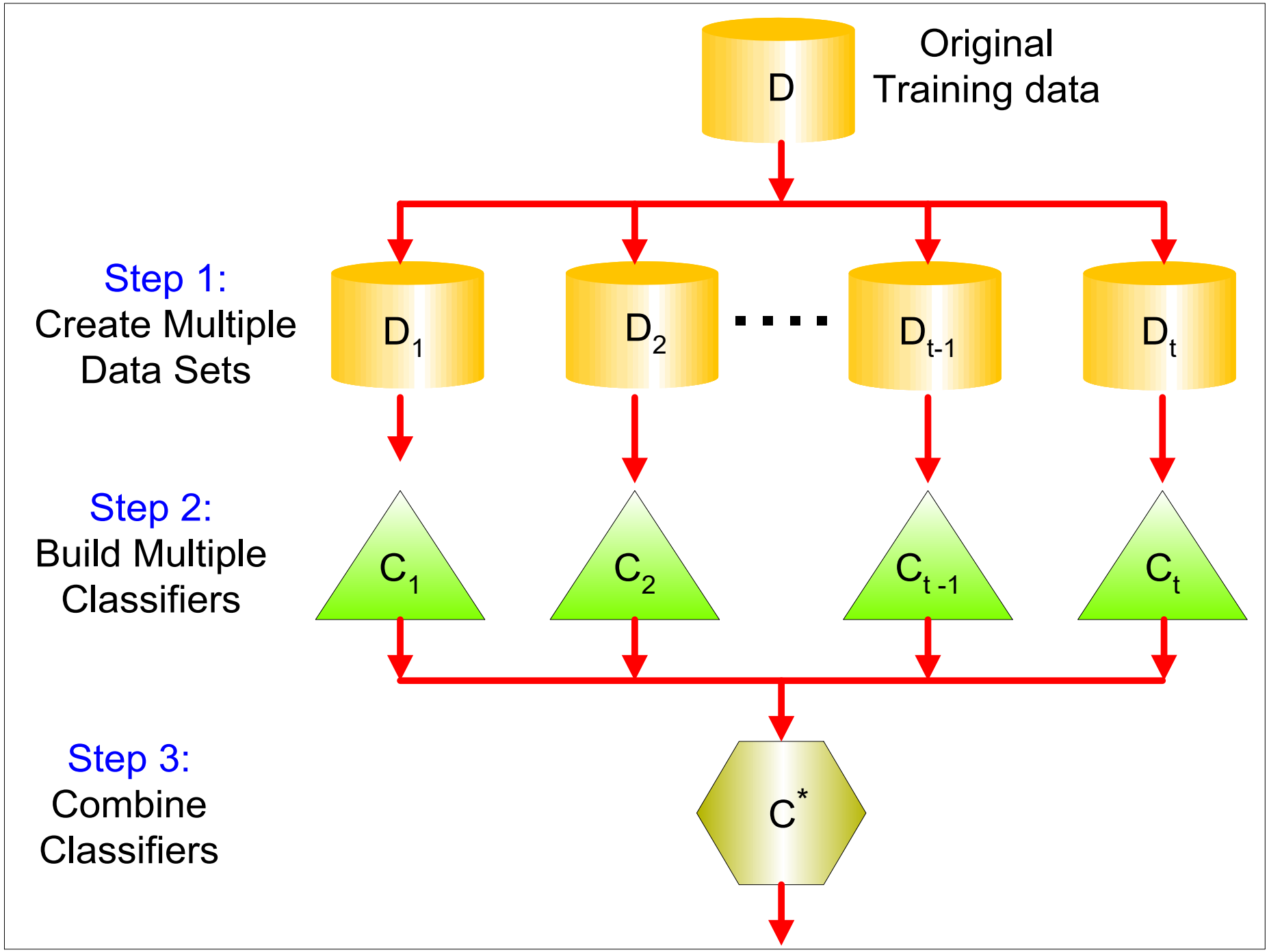
Requires N(N - 1)/2 classifiers

Ensemble Methods

Bagging, Boosting, Adaboost, Random Forest

Ensemble Classifier Concept

- Two heads are better than one
- Constructs a set of base classifiers from training data
- An ensemble combines multiple base classifiers to make a final prediction.
  ① Hard voting: majority vote of predicted labels
  ② Soft voting: average (or weighted average) class probabilities
  ③ Weighted voting: give more reliable classifiers larger weights


Original
Training data
D
Step 1:
Create Multiple
Data Sets
D1
D2
Dt-1
Dt
Step 2:
Build Multiple
Classifiers
C1
C2
Ct -1
Ct
Step 3:
Combine
Classifiers
C*

Ensemble Classifier (cont.)

- Ensemble methods tend to improve performance when:
  1. Base classifiers are diverse (their errors are not highly correlated).
  2. For simple majority voting, base classifiers should perform better than random guessing. (error rate < 0.5 for binary classification)
  - In practice, it is difficult to ensure complete independence among the base classifiers. (Reducing error correlation is often sufficient)

Example: Ensemble Methods

- Suppose there are 25 base classifiers, each classifier has error rate = 0.35
  - Majority vote of classifiers used for classification
  - If all classifiers make identical predictions:
    - Error rate of ensemble = 0.35
  - If all classifier are independent (error are uncorrelated):
    - Error rate of ensemble is the probability of having more than half of base classifiers being wrong
    - $error_{ensemble} = \sum_{i=13}^{25} \binom{25}{i} e^i (1-e)^{25-i} = 0.06$

Bagging (bootstrap aggregating)

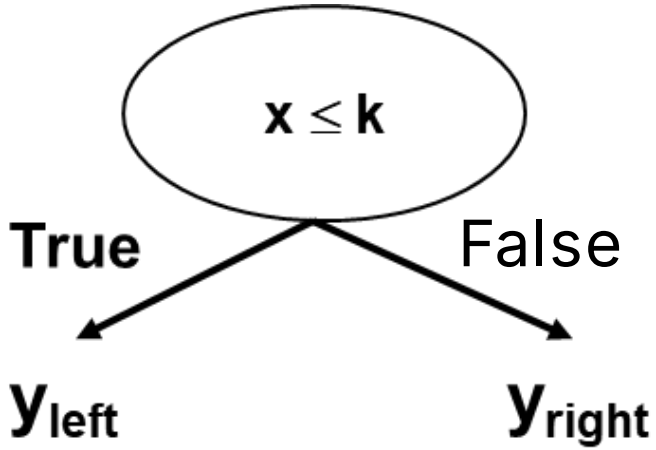
- Repeatedly draw N samples with replacement from the training set to create k bootstrap samples.
  - Each original training instance has probability $1 - \left(1 - \frac{1}{n}\right)^n$ of being selected in each round (if n is sufficiently large, converges to 0.632)
  - Each bootstrap sample has the same size as the original training data

Bagging algorithm

1: Let k be the number of bootstrap samples.
2: **for** $i = 1$ to k **do**
3: Create a bootstrap sample of size N, D_i.
4: Train a base classifier C_i on the bootstrap sample D_i.
5: **end for**
6: $C^*(x) = \underset{y}{\operatorname{argmax}} \sum_i \delta\big(C_i(x) = y\big)$.
$\{\delta(\cdot) = 1$ if its argument is true and 0 otherwise.$\}$

Bagging algorithm (cont.)

- Classifier: a decision stump (a decision tree of depth 1)
- Choose the split point k that maximizes information gain



x	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1
y	1	1	1	−1	−1	−1	−1	1	1	1

Original data

Bagging Round 1:

x	0.1	0.2	0.2	0.3	0.4	0.4	0.5	0.6	0.9	0.9
y	1	1	1	1	-1	-1	-1	-1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 2:

x	0.1	0.2	0.3	0.4	0.5	0.8	0.9	1	1	1
y	1	1	1	-1	-1	1	1	1	1	1

x <= 0.65 ==> y = 1
x > 0.65 ==> y = 1

Bagging Round 3:

x	0.1	0.2	0.3	0.4	0.4	0.5	0.7	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 4:

x	0.1	0.1	0.2	0.4	0.4	0.5	0.5	0.7	0.8	0.9
y	1	1	1	-1	-1	-1	-1	-1	1	1

x <= 0.3 ==> y = 1
x > 0.3 ==> y = -1

Bagging Round 5:

x	0.1	0.1	0.2	0.5	0.6	0.6	0.6	1	1	1
y	1	1	1	-1	-1	-1	-1	1	1	1

x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

Bagging Round 6:

x	0.2	0.4	0.5	0.6	0.7	0.7	0.7	0.8	0.9	1
y	1	-1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 7:

x	0.1	0.4	0.4	0.6	0.7	0.8	0.9	0.9	0.9	1
y	1	-1	-1	-1	-1	1	1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 8:

x	0.1	0.2	0.5	0.5	0.5	0.7	0.7	0.8	0.9	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 9:

x	0.1	0.3	0.4	0.4	0.6	0.7	0.7	0.8	1	1
y	1	1	-1	-1	-1	-1	-1	1	1	1

x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

Bagging Round 10:

x	0.1	0.1	0.1	0.1	0.3	0.3	0.8	0.8	0.9	0.9
y	1	1	1	1	1	1	1	1	1	1

x <= 0.05 ==> y = -1
x > 0.05 ==> y = 1

① x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

② x <= 0.65 ==> y = 1
x > 0.65 ==> y = 1

③ x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

④ x <= 0.3 ==> y = 1
x > 0.3 ==> y = -1

⑤ x <= 0.35 ==> y = 1
x > 0.35 ==> y = -1

⑥ x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

⑦ x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

⑧ x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

⑨ x <= 0.75 ==> y = -1
x > 0.75 ==> y = 1

⑩ x <= 0.05 ==> y = -1
x > 0.05 ==> y = 1

Round	x=0.1	x=0.2	x=0.3	x=0.4	x=0.5	x=0.6	x=0.7	x=0.8	x=0.9	x=1.0
1	1	1	1	-1	-1	-1	-1	-1	-1	-1
2	1	1	1	1	1	1	1	1	1	1
3	1	1	1	-1	-1	-1	-1	-1	-1	-1
4	1	1	1	-1	-1	-1	-1	-1	-1	-1
5	1	1	1	-1	-1	-1	-1	-1	-1	-1
6	-1	-1	-1	-1	-1	-1	-1	1	1	1
7	-1	-1	-1	-1	-1	-1	-1	1	1	1
8	-1	-1	-1	-1	-1	-1	-1	1	1	1
9	-1	-1	-1	-1	-1	-1	-1	1	1	1
10	1	1	1	1	1	1	1	1	1	1
Sum	2	2	2	-6	-6	-6	-6	2	2	2
Sign	1	1	1	-1	-1	-1	-1	1	1	1
True Class	1	1	1	-1	-1	-1	-1	1	1	1

Boosting: Sample Weight

- Boosting trains base classifiers sequentially.
  - Initially, every training example has the same weight
    - $w_i^{(1)} = \frac{1}{N}, i = 1, \ldots, N$
  - After each round, the weights are updated:
    - Misclassified examples receive higher weights.
    - Correctly classified examples receive lower relative weights.
- Therefore, the next classifier focuses more on examples that previous classifiers handled poorly.

Boosting (cont.)

Higher weight ⟹ Higher sampling probability ⟹ more attention in the next round

Original Data	1	2	3	4	5	6	7	8	9	10
Boosting (Round 1)	7	3	2	8	7	9	4	10	6	3
Boosting (Round 2)	5	4	9	4	2	5	1	7	4	2
Boosting (Round 3)	4	4	8	10	4	5	4	6	3	4

- Suppose that example 4 is misclassified by the classifier in Round 2.
- Its weight is increased before Round 3, so it has a higher probability of being sampled multiple times in the next training set.

Bagging: uniform sampling. Classifiers are trained independently.
Boosting: adaptive weighting. Classifiers are trained sequentially.

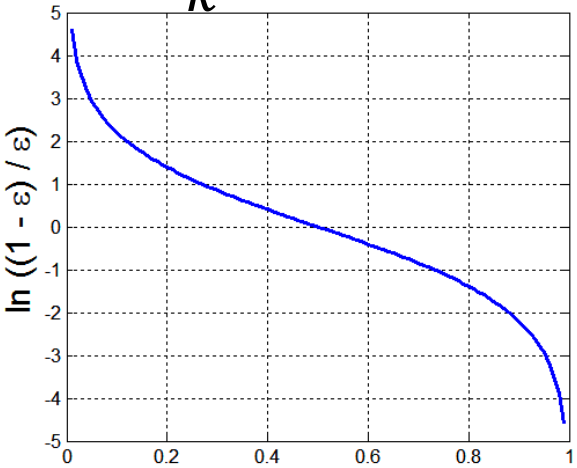
AdaBoost: Classifier Weights

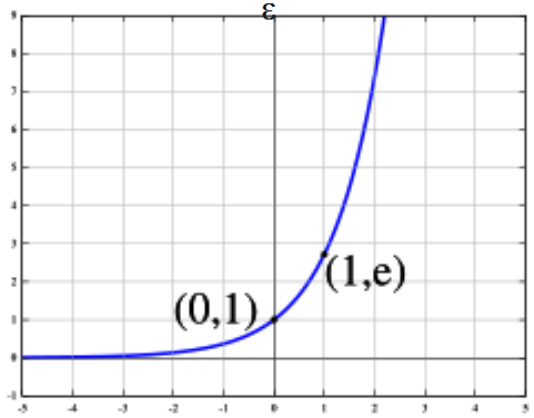
- A specific boosting algorithm that adaptively reweights training examples after each round.
  - Misclassified examples receive larger weights in the next round.
  - More accurate base classifiers receive larger voting weights in the final ensemble

Sample and Classifier Weight

- Suppose the weight in the k round is: $w_1^{(k)}, w_2^{(k)}, \ldots w_N^{(k)}$
- The k round's classifier is δ_k, the error rate of δ_k is:
  - $\varepsilon_k = \frac{1}{N}[\sum_{j=1}^{N} w_j \delta(C_i(x_j) \neq y_j)]$
  - The important of a Classifier $\delta_k, \alpha = \frac{1}{2}\ln(\frac{1-\varepsilon_k}{\varepsilon_k})$
    - $\varepsilon_k < 0.5 \Rightarrow \alpha > 0$: useful classifier, votes positively
    - $\varepsilon_k = 0.5 \Rightarrow \alpha = 0$: no better than random guessing
    - $\varepsilon_k > 0.5 \Rightarrow \alpha < 0$: worse than random guessing
- Sample weight update

$$w_j^{(k+1)} = \frac{w_j^k}{Z_j} \begin{cases} exp^{-\alpha_j} & if\ C_j(x_j) = y_j \\ exp^{\alpha_j} & if\ C_j(x_j) \neq y_j \end{cases}$$





AdaBoost Algorithm

1: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \ldots, N\}$. {Initialize the weights for all N examples.}
2: Let k be the number of boosting rounds.
3: **for** $i = 1$ to k **do**
4: Create training set D_i by sampling (with replacement) from D according to $\mathbf{w}$.
5: Train a base classifier C_i on D_i.
6: Apply C_i to all examples in the original training set, D.
7: $\epsilon_i = \frac{1}{N}\big[\sum_j w_j \, \delta\big(C_i(x_j) \neq y_j\big)\big]$ {Calculate the weighted error.}
8: **if** $\epsilon_i > 0.5$ **then**
9: $\mathbf{w} = \{w_j = 1/N \mid j = 1, 2, \ldots, N\}$. {Reset the weights for all N examples.}
10: Go back to Step 4.
11: **end if**
12: $\alpha_i = \frac{1}{2} \ln \frac{1-\epsilon_i}{\epsilon_i}$.
13: Update the weight of each example according to Equation 4.103.
14: **end for**
15: $C^*(\mathbf{x}) = \underset{y}{\operatorname{argmax}} \sum_{j=1}^{T} \alpha_j \delta(C_j(\mathbf{x}) = y)\big)$.

Random Forest

- Random Forest is an ensemble of decorrelated decision trees that improves generalization performance.
  - Bootstrap sampling of training examples as in Bagging
  - Random feature selection at each split
- For each tree:
  1. Draw a bootstrap sample from the training set.
  2. Grow an unpruned decision tree.
  3. At each internal node:
     - Randomly select m features from the p available features.
     - Choose the best split only among these m features.
  4. Stop when a node is pure or contains fewer than the minimum number of samples.

Why Random Features?

- A single, strong feature is often chosen as the root split across all trees, resulting in highly correlated trees.
- Each node considers only m random features, leading to more diverse trees and decreased correlation.

Next Week

- Practical Issues in Classification
  - Feature Selection and Feature Engineering
  - Overfitting and Model Complexity
  - Model Evaluation and Selection
  - Class Imbalance